

Luiss

Libera Università Internazionale
degli Studi Sociali Guido Carli

Expressions and Data Types

Introduction to Computer Programming

Management and Artificial Intelligence

LUISS



Why Python?

- A **high-level** language for **beginners**
- with a **clean syntax**, but...
- **Industrial strength** programming language used at thousands of companies (one of three official Google languages)
- Free, well documented, well supported

Recall: We will use Python 3 (there are non-backwards compatible differences with respect to Python 2). Further, Python 2 is now unsupported.

Expressions and Statements

A (Python) program is composed by a sequence of:

- **definitions** (also **expressions**) which are **evaluated**
- **statements** (also **commands**) which are **executed**

In plain terms...

- **expressions** yield some results (e.g., `5 * 5`)
- **statements** instruct the interpreter to do something (e.g., print on screen, assign to variable)

Expressions and statements can either

- be directly written into a shell (**interactive mode**)
- be written into a text file and then loaded into the shell (**script mode**).

Expressions vs. Statements

- **Expressions: Represent** something

- Python *evaluates* expressions
- end-result is a value

Examples:

```
>>> 2.3          # a plain literal
2.3
>>> (7*3+1)-4    # 4 literals and some operators
18
```

- **Statements: Do** something

- Python *executes* statements
- no need to return a value

Examples:

```
>>> print("Hi there!")
Hi there!
>>> import sys
```

Interactive vs. Script Mode

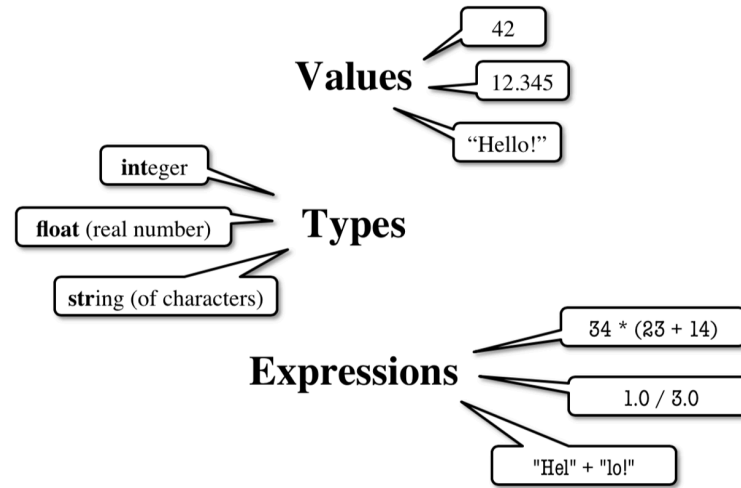
- **Interactive Mode:** perfectly fine for testing and learning.

Type your commands/expressions into the shell and Python will execute/evaluate those immediately:

<code>>>> 5 * 5</code>	←input
<code>25</code>	←output

- **Script Mode:** is what you normally do.
 - write the entire source code in a text file ending with the `.py` extension (say `myscript.py`)
 - `myscript.py` will be executed in its entirety by the Python interpreter

Programming in Python



- programs manipulate **data objects**
- each data object has its own **type**, which defines the type of operations that can be done with them
- objects can either be:
 - **Scalar**: cannot be subdivided
 - **Non-scalar**: have an inner structure that can be accessed

Scalar Objects

- `int`: represents integers (e.g., `5`)
- `float`: represents real numbers (e.g., `3.14`)
- `bool`: represents Boolean values (`True` or `False`)
- `NoneType`: special with only one value (`None`)

Hint: Use the `type()` function if you are not sure about the data type.

Examples:

```
>>> type(109)    ←input  
<class 'int'>    ←output
```

```
>>> type(10.9)   ←input  
<class 'float'> ←output
```

Type `int()`

Values: $\dots, -3, -2, -1, 0, 1, 2, 3, \dots$

Integer literals look like a sequence of numbers with no periods nor commas.

Operations:

- `+`, `-`, `*`, `/`, `//` (floor division), `%` (modulus, remainder), `**` (exponentiation)
- `-` (unary minus, to negate a number)

Examples:

<code>>>> 7/5</code>	←input
<code>1.4</code>	←output
<code>>>> 7//5</code>	← <code>//</code> is the floor division!
<code>1</code>	←output

Note In Python 3, operations between integers may not yield an integer.

Type `float()`

Values: some (approximated) range of $\mathbb{R}$

Note that:

- numbers with a "." are decimals (e.g., `2.0`)
- numbers with no decimal separator are integers (e.g., `2`)

Operations:

- `+`, `-`, `*`, `/`, `%` (modulus, remainder of division)
- `**` (exponentiation)
- `-` (unary, invert sign)

Operations between floats or between a float and an integer, always yield a float.

```
>>> 7.0/5.0 ←input
```

```
1.4 ←output
```

```
>>> 7.0//5 ← // is the floor division!
```

```
1.0
```

Python stores floating point numbers as base 2 (binary) fractions:

$$\{\text{number}\} = \{\text{integer mantissa}\} \cdot 2^{-\{\text{exponent}\}}$$

For example: $0.125 = 1 \cdot 2^{-3} = 1 \cdot \frac{1}{2^3} = 1 \cdot \frac{1}{8}$.

Warning: Not all numbers can be represented in base 2 fractions. Python chooses the best approximation it can. The same also holds for some numbers in base 10 fraction: e.g., what about $1/3$?

Python may yield some **approximation errors**, that propagate as expressions go through.

Example:

>>> 0.1+0.2	←input
0.30000000000000004	←output
>>> 0.1+0.2+0.1+0.2	←input
0.6000000000000001	←output
>>> 0.1+0.2+0.1+0.2+0.1+0.2+0.1+0.2	←input
1.2000000000000002	←output

Type `bool`

Values:

`True` and `False` (capitalized!)

Operations:

$$\text{not } a = \begin{cases} \text{True} & \text{if } a \text{ is False} \\ \text{False} & \text{if } a \text{ is True} \end{cases}$$

$$a \text{ and } b = \begin{cases} \text{True} & \text{if both } b \text{ and } a \text{ are True} \\ \text{False} & \text{otherwise} \end{cases}$$

$$a \text{ or } b = \begin{cases} \text{True} & \text{if } a \text{ is True or } b \text{ is True} \\ \text{False} & \text{otherwise} \end{cases}$$

Booleans are the output of comparisons between items (e.g., `int` or `float`):

- Order: `i < j`, `i > j`, `i <= j`, `i >= j`
- (In)equality: `i != j`, `i == j`

Truth Table between two Boolean variables

a	b	not a	not b	a and b	a or b
True	True	False	False	True	True
False	False	True	True	False	False
True	False	False	True	False	True
False	True	True	False	False	True

Type Conversions

Switching between `float` and `int` can be performed via **casting** (**explicit** conversion):

- `float(2)` converts value `2` (integer) to `2.0` (floating point)
- `int(2.9)` converts value `2.9` (floating point) to `2` (integer)

Warning `int()` does not round to the nearest integer: rather, it truncates the decimal part. To round to the nearest integer, use `round()` instead.

Python supports **widening** automatically (**implicit** conversion):

$\underbrace{\text{bool} \rightarrow \text{int} \rightarrow \text{float}}_{\text{narrow to wide}}$

- **Widening:** Python does it automatically, if needed. Example:

`1/0.5=2.0` because Python casts `1` to `float`

- **Narrowing:** Python never does it automatically. Because narrowing leads to information loss!

Expressions

- **Expressions** are combinations of **objects** and **operators**
- an expression yields a **value**, which has its own **type**
- the simplest expression has the form:

`<object, operator, object>`

Arithmetic Operations on `int` and `float`: a Recap

Syntax	Operator	Output Type
<code>i+j</code>	Sum	if both of them are floats, result is float
<code>i-j</code>	Difference	if both of them are int, result is int
<code>i*j</code>	Product	if either of them is float, result is float
<code>i/j</code>	Division	result is float (always)
<code>i%j</code>	Remainder (Modulo)	if both of them are floats, result is float
<code>i**j</code>	Power of	if both of them are int, result is int
<code>i//j</code>	Floor division	if either of them is float, result is float

Comparison Between `int`s and `float`s: a Recap

NOTE: Comparisons below evaluate to a `bool`.

Expression	Evaluation
<code>i < j</code>	→ True if <code>i</code> is smaller than <code>j</code>
<code>i <= j</code>	→ True if <code>i</code> is smaller than or equal to <code>j</code>
<code>i > j</code>	→ True if <code>i</code> is greater than <code>j</code>
<code>i >= j</code>	→ True if <code>i</code> is greater than or equal to <code>j</code>
<code>i == j</code>	→ True if <code>i</code> is equal to <code>j</code>
<code>i != j</code>	→ True if <code>i</code> is not equal to <code>j</code>

Chained Comparisons

Comparisons can be chained arbitrarily and comparisons have the same priority:

$i < j < k$ is the same as $i < j$ and $j < k$

Since comparisons do not act in a pairwise fashion, *weird* comparisons like

$i < j > k$

are perfectly valid since it is equivalent to $(i < j)$ and $(j > k)$.

Operator Precedence

Precedence between operators can be *forced* via parenthesis.

In principle, operators follow the **PEMDAS** order:

1. **P**arenthesis
2. **E**xponential
3. **M**ultiplication and **D**ivision
4. **A**ddition and **S**ubtraction

The Big Picture of Operator Precedence

1. Parenthesis: `(...)`
2. Exponentiation: `**`
3. Unary operators: `+` and `-`
4. Binary arithmetic: `*`, `/`, `//` and `%`
5. Binary arithmetic: `+` and `-`
6. Comparisons: `>`, `<`, `>=`, `<=`, `==`, `!=`
7. Logical `not`
8. Logical `and`
9. Logical `or`

NOTE: Same line means same precedence. Read ties left to right.

HINT: In case of emergency, use parenthesis!

